



UNIVERSITÀ
POLITECNICA
DELLE MARCHE

Università Politecnica delle Marche

Facoltà di Ingegneria Informatica e dell'Automazione

Chatbot

Ricerca e consigli di videogiochi vintage

Autori:

Matteo Sonaglioni

Enzo Cingoli

Gabriel Vladut Popa

Anno Accademico 2025–2026

Relazione redatta per il corso di Data Science

Indice

1	Introduzione	3
1.1	Definizione e Ruolo dei Chatbot	3
1.2	Il Framework Rasa	3
2	Dataset	5
2.1	Descrizione del dataset	5
2.2	ETL	6
2.3	Progettazione degli intenti	8
2.4	Identificazione delle entità e gestione degli slot	9
2.5	Definizione delle risposte e del dominio	10
2.6	Implementazione delle Actions e Logica Applicativa	11
2.7	Gestione del Dialogo e Logica Conversazionale (Rules)	15
2.7.1	Gestione della Sessione e Interazioni Base	15
2.7.2	Orchestrazione Avanzata del Form (game_preferences_form)	15
2.7.3	Gestione dei "No Results" e Comandi Diretti	16
2.8	Pipeline di Comprensione del Linguaggio Naturale	16
2.8.1	Componenti di Tokenization e Featurization (Preparazione del Testo)	16
2.8.2	Classification e Entity Recognition (Modelli di Apprendimento)	17
2.8.3	Robustezza e Fallback	17

1 Introduzione

Il presente elaborato si pone l'obiettivo di illustrare le fasi di progettazione e sviluppo di un ChatBot dedicato alla ricerca e alla raccomandazione di videogiochi vintage pubblicati nell'arco temporale compreso tra il 2004 e il 2010 su varie piattaforme. Questo periodo storico rappresenta una fase cruciale per l'industria del videogioco, caratterizzata dalla transizione verso l'alta definizione e dalla maturazione delle meccaniche di gioco moderne; tuttavia, l'accessibilità alle informazioni riguardanti i titoli meno noti di tale epoca rischia di perdersi nella vasta mole di dati odierni. Il ChatBot mira a colmare questo divario, offrendo un'interfaccia che faciliti il recupero di informazioni e guidi l'utente nella riscoperta di opere vintage, preservando così la memoria storica di un segmento significativo del patrimonio videoludico.

1.1 Definizione e Ruolo dei Chatbot

Nel panorama dell'interazione Uomo-Macchina i ChatBot rappresentano una delle applicazioni più tangibili dell'Intelligenza Artificiale, definiti come software in grado di simulare una conversazione intelligente con uno o più interlocutori umani tramite modalità uditive o testuali. Questi sistemi operano attraverso l'elaborazione del linguaggio naturale (NLP), che permette alle macchine di comprendere, interpretare e generare linguaggio umano in modo contestualmente appropriato. L'adozione di interfacce per la conversazione segna un cambio di paradigma rispetto alle tradizionali interfacce grafiche (come le GUI): l'utente, infatti, non deve più apprendere la navigazione di un sistema complesso, ma può esprimere i propri intenti attraverso un linguaggio libero, delegando all'agente il compito di tradurre tali richieste in azioni operative e query di ricerca strutturate.

1.2 Il Framework Rasa

Per la realizzazione tecnica dell'agente è stato utilizzato Rasa, un framework open-source per lo sviluppo di assistenti contestuali basati sul Machine Learning. A differenza dei tradizionali approcci basati su regole statiche, Rasa implementa un'architettura modulare composta da *Rasa NLU*, per la comprensione del linguaggio naturale (classificazione degli intenti ed estrazione delle entità), e *Rasa Core*, per la gestione del dialogo. Questa struttura permette al ChatBot di analizzare la richiesta dell'utente e mantenere una memoria del contesto conversazionale, prevedendo l'azione successiva più probabile basandosi su modelli probabilistici addestrati su storie di conversazioni reali.

L'architettura generale del sistema è composta dai seguenti componenti principali:

- **Rasa NLU:** responsabile della classificazione degli intenti e dell'estrazione delle entità dalle frasi dell'utente.
- **Rasa Core:** gestisce il dialogo, mantenendo il contesto conversazionale e determinando l'azione successiva tramite modelli probabilistici.
- **Action Server:** ospita le Custom Actions, che permettono di interrogare il dataset dei videogiochi e restituire risposte dinamiche.
- **Dataset dei videogiochi:** struttura dati locale contenente informazioni su titoli, anno di uscita, piattaforma e genere.

Il diagramma a blocchi nella Figura 1 evidenzia la comunicazione tra i componenti, mostrando come l'input dell'utente venga processato e trasformato in una risposta coerente.

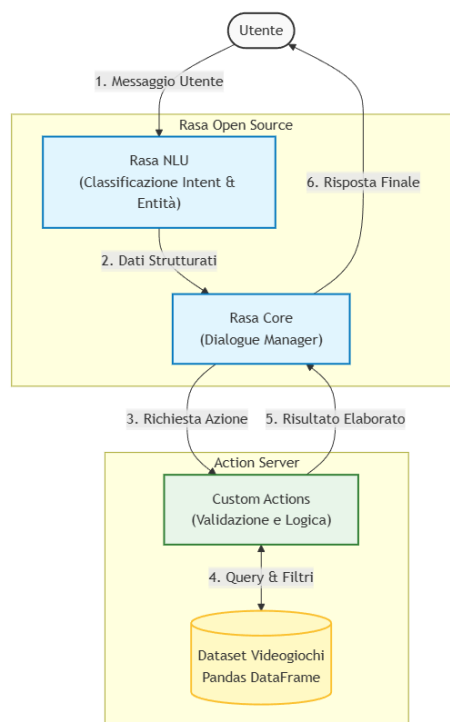


Figura 1: Diagramma a blocchi raffigurante il sistema Rasa messo in comunicazione con il dataset.

2 Dataset

Per popolare la base di conoscenza del Chatbot e garantire risposte accurate e pertinenti, è stato scelto il **Managerial and Decision Economics 2013 Video Games Dataset** distribuito dalla *University of Portsmouth*. La scelta è motivata dalla presenza granulare di metadati essenziali. Infatti, il dataset contiene **1770** titoli per 166 variabili e comprende non solo metadati anagrafici (titolo, console, anno), ma anche una vasta gamma di *feature binarie* che descrivono nel dettaglio le meccaniche di gioco, i sottogeneri e le caratteristiche strutturali di ogni titolo (ad esempio "SideScrolling", "Isometric", "Puzzle"). Questa ricchezza di attributi permetterà al Chatbot di andare oltre la semplice ricerca per titolo, abilitando query complesse basate sul contenuto e sullo stile di gioco. Inoltre, l'inclusione dei dati di vendita globali offre una metrica oggettiva per definire la "popolarità" di un titolo, parametro cruciale per l'algoritmo di raccomandazione del sistema, che potrà quindi suggerire non solo titoli di nicchia, ma anche i *best-seller* che hanno definito quell'era videoludica.

2.1 Descrizione del dataset

Il dataset (Tabella 1) presenta una struttura complessa, caratterizzata dalla presenza massiccia di variabili categoriali codificate in formato binario (*One-Hot Encoding*). Le principali tipologie di dati disponibili includono:

- **Dati anagrafici:** Title, Platform e YearReleased;
- **Metriche economiche:** vendite nel mercato statunitense e variabili econometriche derivate (blocchi temporali di vendita);
- **Publisher:** una serie di colonne binarie indicanti la casa di produzione;
- **Caratteristiche di gameplay (Tags):** oltre 100 colonne booleane che fungono da tag descrittivi, spaziando dai macro-generi (Action, Strategy) a micro-meccaniche specifiche (Wakeboarding, Volleyball, Tricks) e prospettive di gioco (FirstPerson, ThirdPerson).
- **Età consigliata:** variabili indicatrici per la classificazione d'età (RatingE, RatingT, RatingM);
- **Altre informazioni generali:** includono la presenza o meno di un Sequel, il prezzo (Usedprice), il Review Score, il numero massimo di giocatori, la presenza dell'online o se il gioco è multiplatforma.

Tabella 1: Estratto del dataset: prime 5 osservazioni con selezione delle variabili principali.

Console	Title	US Sales	Year	...	Platform	Isometric	...	ThirdPerson
Nintendo DS	Super Mario 64 DS	4.69	2004	...	1	0	...	1
Sony PSP	Lumines: Puzzle Fusion	0.56	2004	...	0	0	...	0
Nintendo DS	WarioWare Touched!	0.54	2004	...	1	1	...	1
Sony PSP	Hot Shots Golf: Open Tee	0.49	2004	...	0	0	...	1
Nintendo DS	Spider-Man 2	0.45	2004	...	1	0	...	1

Nota: La colonna "..." rappresenta le variabili omesse (metriche di blocco, anni specifici e altri tag di genere) per brevità.

2.2 ETL

Per ottimizzare le prestazioni del modello NLP e ridurre il rumore informativo, è stata implementata una **pipeline ETL** in Python utilizzando la libreria *Pandas*, articolata come segue:

- **Extract e Pre-processing:** l'ingestione dei dati ha richiesto una gestione specifica della codifica caratteri (**latin-1**) per preservare la correttezza dei titoli contenenti caratteri speciali. In questa fase è stata verificata la consistenza strutturale del dataframe.
- **Transform e Feature Engineering:** questa fase è il cuore dell'elaborazione, destinandolo all'analisi econometrica in uno adatto all'interazione con il ChatBot:
 - **Consolidamento dei Publisher:** le molteplici colonne on-hot relative agli editori sono state decodificate e condensate in un'unica variabile categoriale nominale **Publisher**, riducendo drasticamente la sparsità della matrice.
 - **Dimensionality Reduction:** sono state rimosse variabili ridondanti o non funzionali all'esperienza utente, incluse le metriche econometriche complesse (Block4, YearReleasedSq, lnUsedPrice) e le colonne temporali specifiche, non necessarie per l'interfaccia utente.
 - **Aggregazione semantica dei generi (Tagging):** è stata implementata una logica di aggregazione per i tag di gioco. Attraverso l'iterazione sulle colonne comprese tra "Action" e "ThirdPerson", è stata generata una nuova feature sintetica denominata **Subgenres**. Questa colonna contiene una stringa concatenata di tutti i tag attivi per il gioco (es.

"Action|Racing|Sports|Anime"), permettendo al Chatbot di parsare facilmente le caratteristiche multiple di un titolo in un'unica operazione di lettura.

- **Normalizzazione del Rating:** le variabili binarie relative al rating ESRB (RatingE, RatingT, RatingM) sono state convertite in un unico attributo **AgeRating** ('E', 'T', 'M' o 'Unknown'), facilitando il filtraggio dei contenuti per fascia d'età.
- **Load:** il risultato finale è un dataset "pulito" (Tabella 2) ed essenziale per garantire la massima leggibilità da parte del modulo di *Data Ingestion* di Rasa.

Tabella 2: Campione del dataset pulito (*vintage_games_clean.csv*): selezione delle variabili principali.

Console	Title	US Sales	Year	Publisher	Genre	...	AgeRating
Nintendo DS	Super Mario 64 DS	4.69	2004	Nintendo	Action	...	E
Sony PSP	Lumines: Puzzle Fusion	0.56	2004	Ubisoft	Strategy	...	Unknown
Nintendo DS	WarioWare Touched!	0.54	2004	Nintendo	Action, Racing / Driving, Sports	...	E
Sony PSP	Hot Shots Golf: Open Tee	0.49	2004	Sony	Sports	...	Unknown
Nintendo DS	Spider-Man 2	0.45	2004	Activision	Action	...	E

Nota: La colonna "... " rappresenta le variabili omesse per brevità (SubGenres, Review Score, UsedPrice, Sequel, ecc.).

2.3 Progettazione degli intenti

Gli *intent*, definiti all'interno del file `domain.yml`, rappresentano le categorie semantiche delle richieste dell'utente e sono fondamentali per mappare il linguaggio naturale verso azioni specifiche del ChatBot. Per il sistema di raccomandazione videoludica sono stati definiti i seguenti intenti principali:

- **greet**, **goodbye** e **help**: gestiscono le fasi di apertura e chiusura della conversazione, oltre a fornire assistenza generica sulle funzionalità del bot.
- **request_form**: identifica la richiesta esplicita dell'utente di ricevere un suggerimento, attivando la procedura di raccolta dati.
- **inform**: permette all'utente di fornire dettagli specifici sulle proprie preferenze (come console, genere, budget, ecc.); è l'intento cardine per popolare gli *slot* del sistema.
- **submit** e **search**: confermano la volontà dell'utente di procedere con la ricerca ed visualizzare i risultati sulla base dei criteri forniti.
- **deny**: gestisce i rifiuti o la volontà dell'utente di correggere un'informazione precedentemente inserita.
- **stop_searching** e **reset**: permettono rispettivamente di interrompere il flusso di ricerca corrente o di azzerare tutte le preferenze per ricominciare la selezione da capo.
- **filters**: utilizzato quando l'utente specifica quali vincoli o filtri desidera rilasciare (utile in scenari di ricerca troppo restrittiva).
- **ask_best_sellers** e **ask_highest_rated**: gestiscono query dirette indipendenti dal form di raccomandazione, per ottenere rispettivamente i giochi più venduti o quelli con le valutazioni migliori.
- **save_game**, **delete_save_games** e **show_saved_games**: implementano la funzionalità di *wishlist*, consentendo all'utente di salvare o eliminare un titolo specifico, e di visualizzare la lista dei giochi precedentemente salvati.

Per ciascun intento sono state raccolte frasi di addestramento rappresentative, poi inserite nel file `nlu.yml`, ottenute sia dal dataset sia dalla formulazione libera di possibili query degli utenti. Questo approccio consente al modello NLU di generalizzare a nuove frasi simili.

2.4 Identificazione delle entità e gestione degli slot

Le entità, anch'esse definite nel `domain.yml`, permettono di estrarre parametri specifici dal linguaggio naturale, i quali vengono successivamente memorizzati in *slots* (variabili di contesto) per filtrare il dataset e restituire raccomandazioni pertinenti. Ogni slot è caratterizzato da un tipo (testo, float, booleano o lista) e da una mappatura che definisce l'estrazione del valore. La configurazione prevede:

- **Caratteristiche descrittive (Testo):** Questi slot vengono popolati tramite mappatura diretta (`from_entity`) dall'intento *inform*:
 - **console:** la piattaforma di gioco richiesta.
 - **genre:** la categoria o il genere del videogioco.
 - **publisher:** l'editore del videogioco.
 - **age_rating:** la classificazione per età (es. PEGI o ESRB).
 - **game_title_to_save:** slot temporaneo che cattura l'entità *game_title* per le operazioni di salvataggio.
- **Parametri numerici (Float):** Gestiti all'interno del *game_preferences_form*, questi slot acquisiscono l'intero input testuale dell'utente (`from_text`) durante la richiesta specifica del form:
 - **release_year:** anno di pubblicazione.
 - **used_price:** budget massimo o prezzo dell'usato desiderato.
 - **review_score:** punteggio minimo di recensione richiesto.
 - **max_player:** numero massimo di giocatori supportati.
- **Funzionalità specifiche (Booleani):** Anch'essi gestiti tramite input testuale nel form per confermare o smentire una caratteristica:
 - **online:** presenza di funzionalità di gioco in rete.
 - **multiplatform:** disponibilità del titolo su più piattaforme contemporaneamente.
- **Gestione dello stato e liste (Lista):** Slot complessi utilizzati per la logica di controllo e la persistenza dei dati:
 - **last_result:** slot tecnico (mappatura *custom*) per memorizzare temporaneamente i giochi trovati.

- **saved_games**: lista persistente (mappatura *custom*) che contiene i titoli salvati dall’utente nella propria "wishlist".
- **filters_to_relax**: lista popolata dall’entità omonima, utilizzata quando l’utente decide quali vincoli rimuovere in caso di ricerca infruttuosa.

2.5 Definizione delle risposte e del dominio

Le risposte, collocate all’interno del file `domain.yml`, sono state strutturate in quattro categorie funzionali principali, rispecchiando la maggiore complessità del flusso di dialogo:

- **Interazione e Assistenza**: Gestiscono il ciclo di vita della conversazione. Oltre a **utter_greet** (“Hi! I’ll be your Retro Gaming Assistant...”) e **utter_goodbye**, è stata introdotta la risposta **utter_help**. Quest’ultima funge da menu di navigazione, illustrando le capacità del bot: ricerca giochi, consultazione classifiche (“Top Charts”), gestione della collezione personale (“save game”) e impostazioni. Include anche **utter_no_understanding** per gestire i fallback NLU.
- **Acquisizione delle Preferenze (*Slot Filling*)**: Il processo di raccolta dati inizia con **utter_start_recommendation** e prosegue con domande mirate per ogni entità. Il sistema interroga l’utente su console (**utter_ask_console**), genere (**utter_ask_genre**), e vincoli numerici come l’anno (**utter_ask_release_year**, range 2004–2010), il budget (**utter_ask_used_price**) e il punteggio minimo (**utter_ask_review_score**). Sono incluse anche domande su booleani (online, multiplatform) che prevedono l’opzione esplicita di salto (“skip”).
- **Validazione e Gestione Errori**: Per garantire la coerenza dei dati, ogni slot dispone di una specifica risposta di errore che interviene quando l’input viola i vincoli del dominio. Esempi includono **utter_release_year_invalid**, **utter_review_score_invalid** (0–100), **utter_max_player_invalid** (1–8) e **utter_age_rating_invalid**. Queste risposte guidano l’utente verso la corretta formulazione del dato richiesto.
- **Gestione del Flusso e Risultati**: Questa categoria gestisce la logica avanzata di navigazione:
 - **Conferma**: **utter_confirm_preferences** riepiloga dinamicamente tutti i parametri inseriti (es. {**publisher**}, {**online**}) prima della ricerca.

- **Risultati:** `utter_submit` avvia la query, mentre `utter_no_results` gestisce l'assenza di match, innescando il flusso di rilassamento dei vincoli tramite `utter_which_filters_to_relax`.
- **Controllo:** `utter_stop_searching` e `utter_ask_to_reset` permettono all'utente di interrompere il form o resettare le preferenze in qualsiasi momento.

2.6 Implementazione delle Actions e Logica Applicativa

Le *custom actions*, implementate nel file `actions.py`, costituiscono il nucleo logico del ChatBot e ne governano il comportamento applicativo. Tali classi estendono le interfacce fornite dal Rasa SDK (in particolare `Action` e `FormValidationAction`) e consentono l'esecuzione di codice Python arbitrario per la validazione degli input, l'interrogazione del dataset e la gestione dello stato conversazionale persistente.

L'infrastruttura si basa sull'uso della libreria **Pandas** per il caricamento e la manipolazione in memoria del dataset CSV (`vintage_games_clean.csv`), caricato una sola volta all'avvio dell'action server, e sulla libreria **Difflib** per la gestione robusta del linguaggio naturale impreciso (*fuzzy matching*). Le principali azioni sono organizzate come segue:

- **Validazione dei dati (`ValidateGamePreferencesForm`):** Questa classe estende `FormValidationAction` ed è responsabile della validazione semantica e sintattica degli slot raccolti durante il form.
 - Per gli slot categoriali (es. *console*, *genre*, *publisher*, *age_rating*), il sistema costruisce dinamicamente una mappa dei valori univoci presenti nel dataset e utilizza `difflib.get_close_matches` per effettuare *fuzzy matching*. Ciò consente di correggere automaticamente errori di battitura e sinonimi (es. “nintndo” → “Nintendo”), notificando all'utente l'eventuale normalizzazione applicata.
 - Per lo slot *genre*, il matching viene esteso anche ai *subgeneri*, consentendo una maggiore flessibilità espressiva.
 - Gli slot numerici (es. *used_price*, *release_year*, *review_score*, *max_player*) vengono validati tramite espressioni regolari per l'estrazione dei valori dal testo libero. Sono inoltre applicati vincoli di dominio (es. anno di uscita compreso tra 2004 e 2010, punteggio tra 0 e 100).

- Tutti gli slot supportano una logica esplicita di *skip*, permettendo all’utente di escludere selettivamente determinati filtri durante la compilazione del form.
- Gli slot booleani complessi (*online* e *multiplatform*) vengono normalizzati interpretando espressioni linguistiche naturali (es. “offline”, “exclusive”, “multi”) in valori simbolici coerenti con il dataset.
- **Motore di Ricerca (`action_search_game`):** È l’azione centrale del sistema. Recupera tutti gli slot validati e applica una sequenza di filtri incrementali al DataFrame Pandas. La ricerca integra logiche eterogenee:
 - *Match parziale* su stringhe testuali (es. console, genere, publisher).
 - *Confronti numerici* per prezzo massimo, punteggio minimo e numero di giocatori.
 - *Filtri booleani* per funzionalità online, multiplatform ed esclusività.
 - *Filtri esatti* per l’age rating.

I risultati vengono paginati (5 elementi per pagina) e memorizzati nello slot `last_result` per consentire la navigazione successiva tramite l’azione dedicata. In assenza di risultati, il sistema innesca un’azione di fallback informativa.

- **Gestione del Flusso e dei Filtri (`action_relax_filters`, `action_reset_game_preferences`):** `action_relax_filters` consente di rimuovere dinamicamente uno o più vincoli applicati alla ricerca. Utilizzando una mappa semantica di parole chiave e una funzione di fuzzy matching sugli slot, l’azione traduce il linguaggio naturale dell’utente nel nome tecnico dello slot da azzerare. `action_reset_game_preferences` permette invece il reset completo delle preferenze, riportando il contesto conversazionale allo stato iniziale.
- **Query Statistiche (`action_show_best_sellers`, `action_show_highest_rated`):** Queste azioni eseguono interrogazioni globali sul dataset, indipendenti dalle preferenze dell’utente. In particolare, ordinano il DataFrame rispettivamente per vendite negli Stati Uniti o per punteggio della critica e restituiscono risultati paginati, sfruttando la stessa infrastruttura di navigazione della ricerca standard.
- **Gestione Wishlist (`action_save_game`, `action_delete_saved_game`, `action_show_saved_games`):** Questo gruppo di azioni implementa la persistenza delle preferenze dell’utente all’interno della sessione. Il salvataggio e la cancellazione dei titoli avvengono tramite *fuzzy matching* sul nome del gioco, mentre la

visualizzazione itera sullo slot lista *saved_games*, fornendo un riepilogo dei titoli preferiti.

- **Navigazione dei Risultati (*action next_page*):** Questa azione gestisce la paginazione incrementale dei risultati precedentemente restituiti, utilizzando gli slot *last_result* e *page* per calcolare dinamicamente gli elementi da visualizzare e migliorare l'usabilità del sistema.

All'interno della Figura 2 viene rappresentata la sequenza di azioni impiegate per gestire un generico input "I need an Aztion RPG".

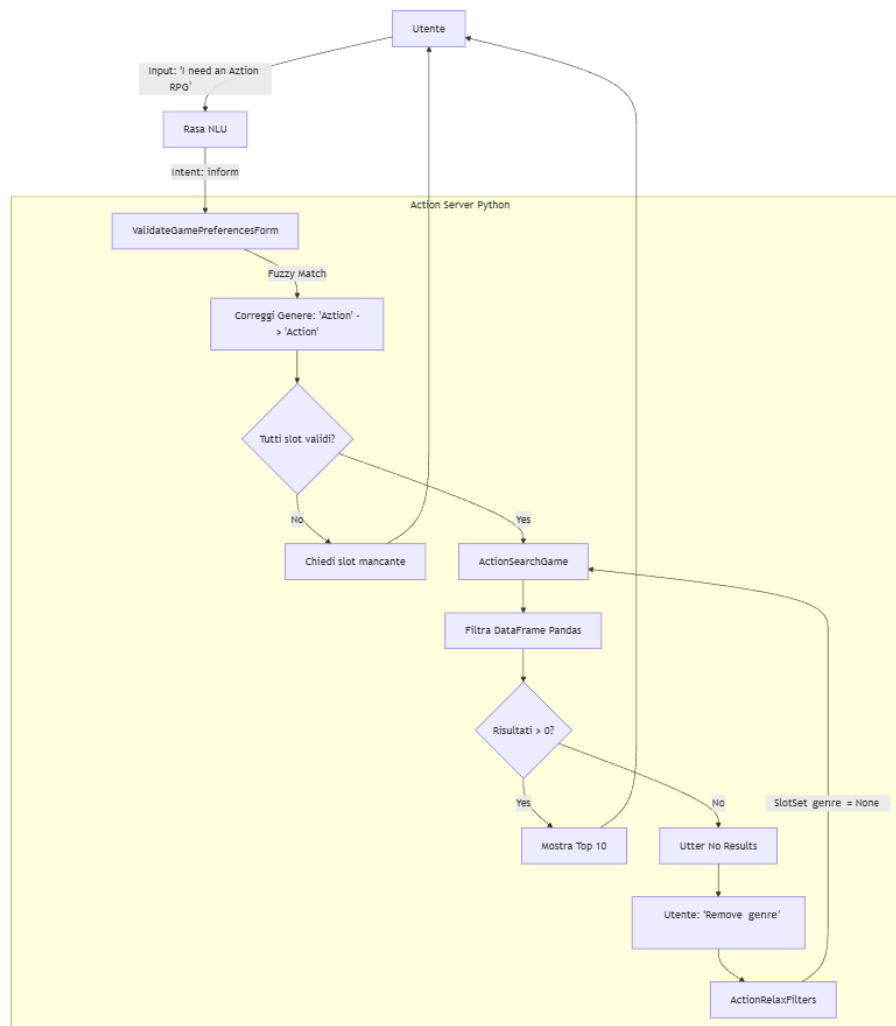


Figura 2: Diagramma del funzionamento generale del Chatbot, attraverso un generico esempio di input.

2.7 Gestione del Dialogo e Logica Conversazionale (Rules)

Le **rules**, definite nel file `rules.yml`, definiscono il comportamento deterministico del chatbot, specificando le azioni da eseguire in risposta a determinati intenti o in condizioni specifiche. Esse sono fondamentali per l'orchestrazione del complesso processo di *slot filling* e per la gestione delle funzionalità "one-shot" (comandi diretti).

2.7.1 Gestione della Sessione e Interazioni Base

Le regole base assicurano una gestione fluida dell'apertura e chiusura, con un focus sull'assistenza immediata all'utente:

- **Ask for help** e **Say goodbye**: Rispondono rispettivamente alle richieste di aiuto e di congedo, terminando la sessione in modo pulito.
- **Fallback**: Gestisce i casi di bassa confidenza NLU (`nlu_fallback`), inviando un messaggio di chiarimento (`utter_no_understanding`) senza interrompere il flusso logico, anche se un form è attivo.

2.7.2 Orchestrazione Avanzata del Form (`game_preferences_form`)

La logica di raccolta dati è stata raffinata per gestire scenari multipli di attivazione e conclusione:

- **Attivazione Ibrida**: Il form può essere avviato esplicitamente (**Start game preferences form** tramite `request_form`) oppure implicitamente. Se l'utente fornisce un dettaglio spontaneo a freddo (`intent: inform`), la regola **Ask to start... by inform** chiede conferma prima di attivare il loop, garantendo un passaggio naturale al processo di raccomandazione.
- **Conclusione e Ricerca**: Esistono due modalità per terminare il form e lanciare `action_search_game`:
 1. **Naturale (After form is filled)**: Quando tutti gli slot obbligatori sono pieni (condizione `requested_slot: null`), il sistema disattiva automaticamente il loop e mostra i risultati.
 2. **Interruzione Volontaria (Search game preferences form)**: Se l'utente invia l'intento `search` ("mostrami i risultati") *durante* la compilazione, il bot interrompe immediatamente la raccolta dati (`action_deactivate_loop`) e procede alla ricerca con i dati parziali acquisiti.

In entrambi i casi, al termine della ricerca viene invocata `action_reset_game_preferences` per pulire il contesto.

- **Gestione del Reset e Stop:** Se l'utente chiede di resettare i filtri mentre il form è attivo, la regola **Ask to reset...** richiede una conferma esplicita per evitare perdite di dati accidentali. Al contrario, l'intento `stop_searching` interrompe definitivamente il processo senza produrre risultati.

2.7.3 Gestione dei "No Results" e Comandi Diretti

Oltre al form, sono state definite regole per gestire gli scenari post-ricerca e le interrogazioni dirette:

- **Rilassamento dei Filtri:** Se la ricerca non produce risultati (`utter_no_results`), una regola specifica attende la conferma dell'utente (`intent: submit`) per chiedere quali vincoli rimuovere (`utter_which_filters_to_relax`). La successiva specifica dei filtri attiva `action_relax_filters` e rilancia la ricerca.
- **Funzionalità One-Shot:** Regole atomiche gestiscono comandi che non richiedono un dialogo multi-turn, eseguendo direttamente le azioni Python corrispondenti:
 - **Show best sellers** → `action_show_best_sellers`.
 - **Show highest rated** → `action_show_highest_rated`.
 - **Save/Show saved games** → Gestione della wishlist tramite `action_save_game` e `action_show_saved_games`.

2.8 Pipeline di Comprensione del Linguaggio Naturale

La pipeline NLU è fondamentale per la trasformazione del testo libero dell'utente in un intento classificato e in entità strutturate. La configurazione adottata segue un approccio ibrido, combinando tecniche basate su *feature* sparse e modelli di *machine learning* profondi, garantendo accuratezza e gestione del contesto.

2.8.1 Componenti di Tokenization e Featurization (Preparazione del Testo)

- **WhitespaceTokenizer:** Scompone il messaggio in singoli *token* in base agli spazi bianchi. L'opzione **case_sensitive: False** rende la classificazione indipendente dalla capitalizzazione.

- **RegexFeaturizer**: Utilizza espressioni regolari (Regex) per identificare *pattern* ricorrenti nel testo (cruciale per estrarre numeri, anni, prezzi, ecc.) e creare *feature* binarie.
- **LexicalSyntacticFeaturizer** e **CountVectorsFeaturizer**: Generano diverse rappresentazioni vettoriali del testo. Il secondo **CountVectorsFeaturizer** utilizza la tecnica **char__wb** (*character-level, word boundaries*) con *n*-grammi da 1 a 4. Questa scelta migliora la resilienza del modello a errori di battitura o variazioni sintattiche, specialmente sui termini del dataset.

2.8.2 Classification e Entity Recognition (Modelli di Apprendimento)

- **DIETClassifier** (*Dual Intent and Entity Transformer*): È il componente centrale che esegue simultaneamente la classificazione degli intenti e l'estrazione delle entità. Viene addestrato per **epochs**: 100 e l'opzione **constrain similarities**: **true** viene utilizzata per migliorare la distinzione tra intenti semanticamente simili.
- **EntitySynonymMapper**: Mappa le diverse espressioni di una stessa entità (es. "DS" o "Nintendo DS") a un unico valore canonico, normalizzando i dati prima della validazione.
- **ResponseSelector**: Gestisce la selezione di risposte predefinite per il *chit-chat* o domande frequenti, operando in parallelo al DIETClassifier.

2.8.3 Robustezza e Fallback

- **FallbackClassifier**: Configurato con **ambiguity_threshold**: **0.1**, questo componente attiva una strategia di *fallback* (richiesta di chiarimenti) se il *confidence score* della classificazione è troppo basso. Questa misura riduce il rischio di fornire risposte errate a fronte di input ambigui o incomprensibili, aumentando la robustezza del dialogo.

La Figura 3 rappresenta un diagramma di flusso della pipeline complessiva.

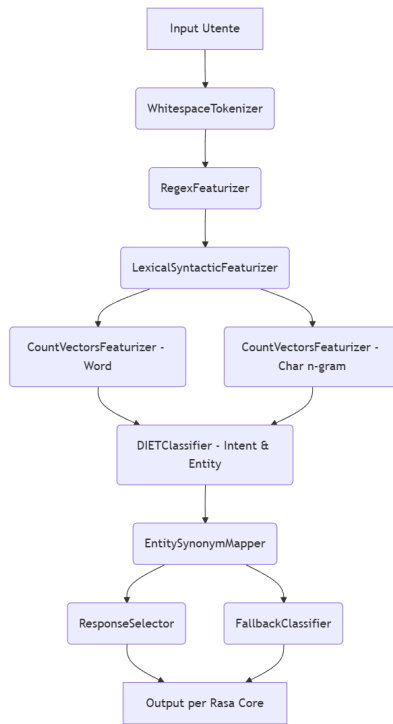


Figura 3: Pipeline NLU